Subject	:	Special Topic in Data Engineering (SECP3843)
Semester	:	2/20252026
Section	:	02
Task	:	Tutorial 3 AI Algorithm
Reference link	:	https://www.youtube.com/watch?v=Rmtr9SY-4VQ
Lecturer	:	Dr. Aryati Binti Bakri
Group	:	1

Prepared by:

No.	Name	Matric Number
1	Lee Yin Shen	A23CS0236
2	Joanne Ching Yin Xuan	A23CS0227
3	Evelyn Goh Yuan Qi	A23CS0222

Table of Content

1.0 What is Image Classification?

Image classification is one of the basic computer vision problem tasks used to classify an image into a number of pre-defined classes according to its visual information. The goal of image classification is to utilize a computer system to automatically recognize and classify the images in terms of features obtained from the input data. This method has been used extensively in various fields, such as medical diagnosis, self-driving cars, facial recognition, security surveillance, and industrial automation.

The CIFAR-10 dataset was used to do the image classification task in this tutorial. The dataset contains 60,000 color images of size 32 x 32 pixels belonging to 10 different classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. During the learning process, the classification model was fed with the data to learn patterns and visual features for each class and to identify the most suitable class for unseen images.

2.0 What is Convolutional Neural Network (CNN) ?

Convolutional Neural Network (CNN) is one of the most popular type of deep learning models used in image processing and computer vision applications. CNN is particularly suitable for the image classification task because it is able to automatically extract useful visual features from images.

A normal Artificial Neural Network, also known as ANN, can process an image by converting it into a one dimensional vector. However, this results in the loss of the spatial relation between pixels in the model. For instance, the model might not grasp the fact that some of the pixels are neighbouring and create a particular pattern. This is a disadvantage in handling image data.

CNN keeps the spatial structure of images, which is the key for overcoming this problem. It scans images with filters by employing convolutional layers. The important patterns that the model can detect with these filters are edges, corners, lines, shapes, textures, and parts of objects. As this image goes deeper and deeper within the layers, CNN can learn more complex features. Table 2.1 shows the main components of CNN and Table 2.2 shows the CNN model used in the tutorial. CNN is more effective than ANN for image classification because it can preserve spatial information and learn visual patterns directly from images.

Table 2.1 Main components of CNN

CNN Component	**Function**
Convolutional Layer	Extracts visual features from the image using filters or kernels.
ReLU Activation	Adds non-linearity so the model can learn complex patterns.
Max Pooling Layer	Reduces the size of feature maps while keeping important information.
Flatten Layer	Converts the feature maps into a one-dimensional vector.
Dense Layer	Uses the extracted features to perform classification.
Softmax Output Layer	Produces probability scores for each class.

Table 2.2 CNN model used in the tutorial

Layer	**Description**
Conv2D	32 filters, 3 × 3 kernel size, ReLU activation
MaxPooling2D	2 × 2 pooling
Conv2D	64 filters, 3 × 3 kernel size, ReLU activation
MaxPooling2D	2 × 2 pooling
Flatten	Converts feature maps into a one-dimensional vector
Dense	64 neurons with ReLU activation
Dense Output	10 neurons with softmax activation

3.0 Evaluation of CNN

The model evaluation is an important part of the machine learning process since it quantifies the performance of a trained model on data it has not yet encountered. The CIFAR-10 test dataset is used to test the performance of the CNN model in this tutorial. The main assessment measures used were accuracy and loss.

Accuracy is the percentage of images classified correctly, out of the total number of images analyzed. The higher the accuracy value, the better the classification. On the other hand, loss value represents the difference between the outputs predicted by the model and the correct class labels. A lower value of the loss means that the model is able to predict the data accurately and has captured the underlying patterns in the dataset more effectively.

The CNN model was trained for 10 epochs. A gradual improvement in the performance of the models was noted during the training. As shown in **Table 3.1**, the training accuracy has gone up from 48.61% in the first epoch to 79.15% in the last epoch. At the same time, the training loss decreased from 1.4403 to 0.5938. The results indicate that the model gradually learned the meaningful image features and gradually enhanced the classification ability during training.

Table 3.1: CNN Training Accuracy and Loss Across 10 Epochs

Epoch	**Training Accuracy**	**Training Loss**
1	0.4861	1.4403
2	0.6156	1.0976
3	0.6634	0.9716
4	0.6923	0.8838
5	0.7176	0.8166
6	0.7373	0.7565
7	0.7510	0.7149
8	0.7669	0.6665
9	0.7810	0.6319
10	0.7915	0.5938

Following the training phase, the CNN model was evaluated using the testing dataset to assess its generalization performance. The evaluation results are presented in Table 3.2.

Table 3.2: CNN Evaluation Results on the CIFAR-10 Test Dataset

Model	**Test Accuracy**	**Test Loss**
CNN	0.7020	0.9337

As presented in **Table 3.2**, the evaluation results indicate that CNN model has test accuracy of 70.20% and test loss of 0.9337. This means that approximately seven out of every ten test images were correctly classified. The accuracy achieved indicated that the CNN model could successfully learn representative visual features from the CIFAR-10 dataset and classify images with satisfactory accuracy.

But there was a gap between the training accuracy (79.15%) and the testing accuracy (69.96%). This difference in performance indicates that there is some moderate overfitting, which means that the model is better at memorizing the training data than it is at predicting new testing data. However, the overall results show that the CNN architecture was able to extract image features and also generated reliable classification results for the CIFAR-10 image classification task.

4.0 Steps hand on in Python

The implementation was carried out using Python with TensorFlow and Keras. The objective of this tutorial was to develop and evaluate an image classification model using the CIFAR-10 dataset. The workflow consisted of dataset loading, preprocessing, ANN implementation, CNN implementation, model evaluation, and prediction.

Step 1: Import Necessary Libraries

The required Python libraries were imported at the beginning of the implementation. TensorFlow and Keras were used to build and train the neural network models. NumPy was used for numerical computations,and Matplotlib was used for image visualization

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 1: Importing Required Python Libraries

Step 2: Load the CIFAR-10 Dataset

The CIFAR-10 dataset was loaded directly from TensorFlow Keras.

```
(X_train,y_train),(X_test,y_test) = datasets.cifar10.load_data()
```

Figure 2: Output of CIFAR-10 Dataset Dimensions

The dataset contains 60,000 coloured images belonging to 10 categories, including airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is divided into 50,000 training images and 10,000 testing images.

The dataset dimensions were examined:

```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
```

Figure 3: CIFAR-10 Dataset Dimensions

Dataset	Shape
X_train	(50000, 32, 32, 3)
X_test	(10000, 32, 32, 3)
y_train	(50000, 1)

Table 4.1: Output of CIFAR-10 Dataset Dimensions

This means the dataset contains 50,000 training images and 10,000 testing images. Each image has a size of 32 × 32 pixels with 3 color channels (RGB).

Step 3: Data Preprocessing

The label arrays are reshaped from two-dimensional arrays into one-dimensional arrays.

```
y_train = y_train.reshape(-1,)
y_test = y_test.reshape(-1,)
```

Figure 4: Label Reshaping and Class Definition

This simplifies label handling during model training and evaluation.The class names are then defined.These labels represent the ten categories contained in the CIFAR-10 dataset.

```
classes = ['airplane','automobile','bird','cat','deer','dog','frog','horse','ship','truck']
```

Figure 5: Class Name

Step 4: Create Image Visualization Function

A helper function named plot_sample() is created to display images together with their corresponding labels.

```
def plot_sample(X,y,index):
    plt.figure(figsize=(15,2))
    plt.imshow(X[index])
    plt.xlabel(classes[y[index]])
```

Figure 6: Function for Visualizing CIFAR-10 Images

The function is then used to display a sample image:

```
plot_sample(X_train,y_train,5)
```

Figure 7: Sample Image from the CIFAR-10 Dataset

This step helps verify that the dataset has been loaded correctly and allows visual inspection of the training images.

Step 5: Feature Normalization

The image pixel values are normalized by dividing them by 255.0.

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Figure 8: Feature Normalization Process

Before normalization, pixel values range from 0 to 255. After normalization, the values range from 0 to 1. This improves training efficiency and helps the model converge more effectively.

Step 6: Build and Train the ANN Model

Before implementing the CNN model, an Artificial Neural Network (ANN) model is developed as a baseline model for comparison.

Table 4.2: ANN Model Architecture

Layer	Description
Flatten	Converts image into one-dimensional vector
Dense	3000 neurons with ReLU activation
Dense	1000 neurons with ReLU activation
Dense Output	10 neurons with softmax activation

```
ann = models.Sequential([
        layers.Flatten(input_shape = (32,32,3)),
        layers.Dense(3000,activation = 'relu'),
        layers.Dense(1000,activation = 'relu'),
        layers.Dense(10,activation = 'softmax')
    ])
```

Figure 9: ANN Model Architecture

The ANN model is created.The model is compiled using the SGD optimizer and sparse categorical crossentropy loss function.

```
ann.compile(optimizer='SGD',
            loss= 'sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Figure 10: ANN Model Compilation

The ANN model is then trained for five epochs.

```
ann.fit(X_train,y_train,epochs=5)
```

Figure 11: ANN Model Training

The training accuracy gradually improves during training and reaches approximately 49%. However, ANN has limitations for image classification because it cannot effectively preserve spatial relationships between pixels.

Step 7: Evaluate ANN Using Classification Report and Heatmap Visualization

The trained ANN model is used to generate predictions on the testing dataset.

```
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]
```

Figure 12: ANN Prediction Generation

A classification report is generated.

```
print('classification report: \n',classification_report(y_test,y_pred_classes))
```

Figure 13: ANN Classification Report

A classification report is generated to evaluate the model using precision, recall, F1-score, and accuracy for each class.The prediction results are also visualized using a heatmap generated with Seaborn.

```
plt.figure(figsize = (14,7))
sns.heatmap(y_pred,annot = True)
plt.ylabel('Truth')
plt.xlabel('Prediction')
plt.title('Confusion matrix')
plt.show
```

Figure 14: ANN Prediction Heatmap Visualization

The heatmap provides a visual representation of the model's prediction outputs and helps identify prediction patterns. The ANN achieved approximately 45% test accuracy, indicating that its performance for image classification is limited compared with the CNN model.

Step 8: Build and Train the CNN Model

After evaluating the ANN model, a Convolutional Neural Network (CNN) model is developed.CNN is more suitable for image classification because convolutional layers can automatically extract spatial features such as edges, textures, shapes, and patterns from images.The CNN architecture consists of two convolutional layers, two max pooling layers, one flatten layer, one dense hidden layer, and one output layer.

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3),activation='relu',input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3),activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Figure 15: CNN Model Architecture

The CNN model is compiled using the Adam optimizer and sparse categorical crossentropy loss function.The model is trained for ten epochs.

```
cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Figure 16: CNN Model Compilation

```
cnn.fit(X_train, y_train, epochs=10)
```

Figure 17: CNN Model Training

Step 9: Evaluate the CNN Model

The CNN model is evaluated using the testing dataset.

```
cnn.evaluate(X_test,y_test)
```

Figure 18: CNN Model Evaluation

The evaluation results are shown below:

Table 4.3: CNN Evaluation Results

Metric	Value
Test Accuracy	70.20%
Test Loss	0.9337

The CNN achieves significantly higher accuracy than the ANN model. This demonstrates that CNN is more effective at learning image features and generalizing to unseen data.

Step 10: Predict Test Images

The trained CNN model is used to predict classes for the testing images.

```
y_pred = cnn.predict(X_test)
y_classes = [np.argmax(element) for element in y_pred]
```

Figure 18: CNN Prediction Generation

Two sample images are examined manually.

Table 4.4: CNN Prediction Results

Test Image Index	**Actual Class**	**Predicted Class**	**Result**
60	Horse	Horse	Correct
100	Deer	Deer	Correct

The predicted labels match the actual labels, indicating that the CNN model can successfully classify unseen images.

In conclusion, the implementation successfully demonstrated the complete workflow of an image classification system using Python, TensorFlow, and Keras. Both ANN and CNN models were developed and evaluated using the CIFAR-10 dataset. The CNN model achieved substantially better performance than the ANN model, obtaining approximately **70.20% test accuracy** compared with approximately **45% test accuracy** achieved by the ANN model. This result demonstrates that CNN architectures are more suitable for image classification tasks because they can automatically extract and learn meaningful spatial features from images.

5.0 What is the weakest in the current CNN model

Although the CNN model achieved substantially better performance than the ANN model, several limitations remain.

First, the current CNN architecture is relatively shallow, consisting of only two convolutional layers and two max-pooling layers. While this structure is sufficient for extracting basic image features, it may not effectively capture complex patterns required to distinguish visually similar classes in the CIFAR-10 dataset, such as cats and dogs.

Second, the model does not employ data augmentation techniques. As a result, training is performed only on the original images, limiting the model's exposure to variations in orientation, position, and scale. This may reduce its ability to generalize to unseen images.

Third, the model lacks regularization mechanisms such as dropout and batch normalization. Without dropout, the model may become overly dependent on specific neurons, increasing the risk of overfitting. Similarly, the absence of batch normalization may lead to less stable training and slower convergence.

Fourth, model evaluation is limited to test accuracy and test loss. While these metrics provide an overall indication of performance, they do not offer detailed insights into class-level prediction behaviour. Additional evaluation metrics such as precision, recall, F1-score, and a confusion matrix would provide a more comprehensive assessment of classification performance.

Finally, the model exhibits signs of mild overfitting. The training accuracy of 79.15% exceeds the test accuracy of 69.96% by approximately 9.19%, indicating that the model performs better on the training data than on unseen test data.

Overall, the primary weaknesses of the current CNN model are its limited depth, lack of data augmentation and regularization techniques, restricted evaluation metrics, and mild overfitting, which collectively constrain its generalization performance.

6.0 How to Enhance the CNN model

An improved model, called new_CNN, can be created to enhance the current CNN model. The proposed enhancement is aimed at increasing the classification accuracy of the model, to avoid overfitting and to increase the generalization capability of the model in the classification of unseen images from the CIFAR-10 test set.

An improvement we can make is to add more convolutional layers. The CNN model that is used now has just two convolutional layers. This architecture can capture the basic visual features of images, but it may not be enough to capture more complex visual patterns like shapes of objects, texture, and fine differences among similar image classes. Adding extra convolutional blocks with more filters, deeper and more meaningful image features can be extracted.

Data augmentation is another improvement that can be used. The original training images are manipulated to produce modified images by techniques known as data augmentation, including random flipping, rotation, zooming, and shifting. The model can be trained to see more variations of the same image without the need for extra data sets using this technique. Consequently the model can be made more powerful and less sensitive to the orientation, position or scale of the original image.

To alleviate overfitting, dropout can be added to the enhanced model as well. The neurons are randomly deactivated during training. This helps to avoid over-fitting the model to a particular characteristic of the training data. Thus, more general patterns can be learned and the model can be improved by applying it to unseen test images.

The enhanced CNN architecture can also be made with batch normalization. It is applied to normalise layers' outputs during training, helping to stabilise the training process.

The model can be trained more smoothly and efficiently by using the batch normalization technique. It can also help enhance convergence and classification performance.

In addition, during the training of the model, the early stopping method can be used. Early stopping is a technique that stops training when the model doesn't improve its performance anymore based on the validation loss. Using this method can avoid over training and unnecessary over fitting.The enhanced **new_CNN** model can therefore include the following improvements.

Table 6.1: new_CNN Improvement

Enhancement	Purpose
Data Augmentation	Creates image variations to improve generalization
Additional Conv2D Layer	Helps the model learn deeper image features
Batch Normalization	Stabilizes and improves the training process
Dropout	Reduces overfitting
Early Stopping	Prevents unnecessary training when performance stops improving
CNN Classification Report	Provides precision, recall, and F1-score
CNN Confusion Matrix	Shows class-level prediction errors

A possible **new_CNN** architecture is shown below:

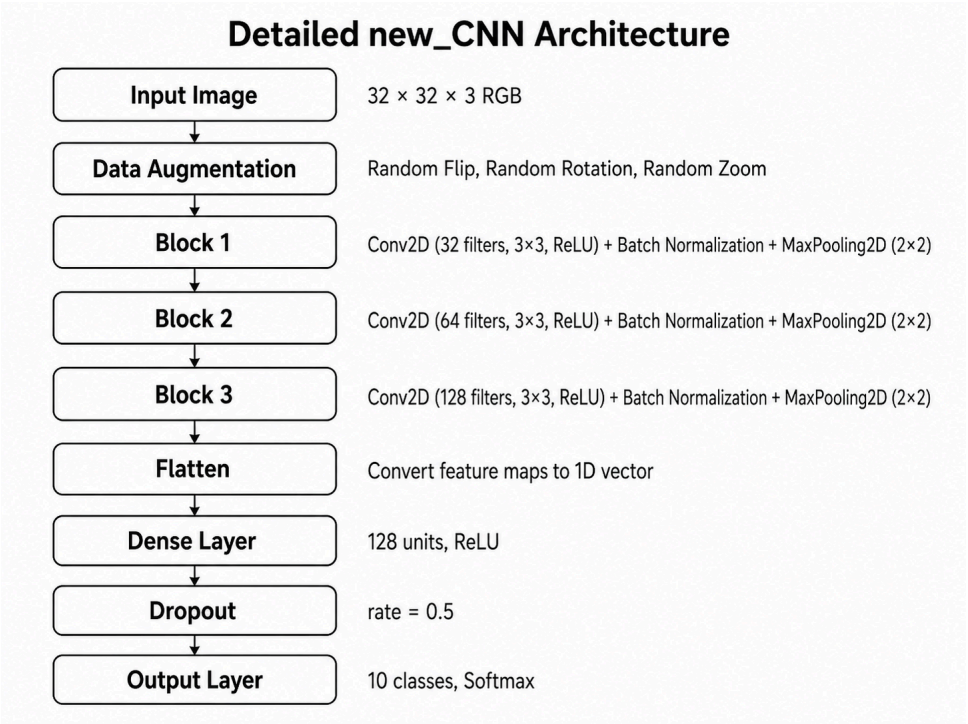

Figure 6.1: new_CNN Architecture

The figure illustrates the proposed **new_CNN** architecture used to enhance the original CNN model. The image first passes through data augmentation to create more training variations, followed by three convolutional blocks with increasing filters to extract simple, medium, and complex image features. Batch normalization helps stabilize the training process, while max pooling reduces the feature map size. After feature extraction, the flatten layer converts the feature maps into a one-dimensional vector, which is then passed to a dense layer for classification. Dropout is added to reduce overfitting, and the final output layer classifies the image into one of the 10 CIFAR-10 classes.

Overall, the enhanced CNN model is expected to perform better than the current CNN model because additional techniques are introduced to improve feature extraction and reduce overfitting. These improvements are expected to help the model classify CIFAR-10 images more accurately and reliably.

7.0 What is the evaluation based on CNN and new_CNN model

The test set for the CIFAR-10 should be used for both the CNN and **new_CNN** models. This is essential so that the comparison between the two models can be made fair and

consistent. Primary evaluations are test accuracy and test loss. The test accuracy is the percentage of images that are correctly predicted by the model, and the test loss is the loss between the predictions and the true class labels.

The original CNN model has a test accuracy of 70.20% and test loss of 0.9337 based on the evaluation result. This means that CNN model correctly classified approximately 70% of images from the test set of CIFAR-10.

The accuracy of the CNN model during training was around 79.15%, and during testing was around 70.20%. The difference between the training and testing accuracy suggests that moderate overfitting may have occurred. This implies that the model has relatively well learned the training data, but not as well the test data.

To overcome the limitations of the original CNN model, an enhanced **new_CNN** model was developed by combining several improvement techniques, including data augmentation, dropout, batch normalization, and additional convolutional layers. These techniques were used to improve generalization, reduce overfitting, and strengthen feature extraction.

Table 7.1: evaluation comparison between CNN and new_CNN

Model	Test Accuracy	Test Loss	Description
CNN	70.20%	0.9337	Baseline CNN model from the tutorial
new_CNN	71.75%	0.8351	Enhanced CNN model with augmentation, dropout, batch normalization, and deeper layers

Based on Table 7.1, the **new_CNN** model achieved better performance than the original CNN model. The test accuracy increased from 70.20% to 71.75%, while the test loss decreased from 0.9337 to 0.8351. This shows that the enhancement improved the model performance, although the improvement was moderate.

Besides accuracy and loss, more detailed evaluation metrics should also be used for **new_CNN**, such as precision, recall, F1-score, and confusion matrix.

Table 7.2: Evaluation metrics for new_CNN

Evaluation Metric	Purpose
Accuracy	Measures the overall correct predictions
Loss	Measures the prediction error
Precision	Measures how many predicted classes are correct
Recall	Measures how many actual classes are correctly detected
F1-score	Balances precision and recall
Confusion Matrix	Shows correct and incorrect predictions for each class

Based on the classification report, an overall accuracy of approximately 72% was achieved by the **new_CNN** model. Better performance was observed for classes such as automobile, ship, frog, truck, airplane, and horse. However, weaker performance was recorded for classes such as cat, bird, deer, and dog. This may be caused by the visual similarity between some animal classes and the low resolution of the CIFAR-10 images.

The sample prediction results also showed that test image index 60 was correctly classified as horse by the **new_CNN** model. However, test image index 100, which belonged to the deer class, was incorrectly predicted as horse. Although this individual prediction was incorrect, the overall test accuracy of the **new_CNN** model remained higher than that of the original CNN model when evaluated using the full test dataset.

In summary, the CNN model was used as the baseline result, while slightly better classification performance was achieved by the **new_CNN** model. The higher test accuracy and lower test loss indicate that the applied enhancement techniques helped improve the generalization ability of the model.

8.0 Results and Discussion

The results indicate that better performance was achieved by the CNN model compared with the ANN model for image classification using the CIFAR-10 dataset. Approximately 45% test accuracy was achieved by the ANN model, while 70.20% test accuracy was achieved by the original CNN model. This shows that CNN is more suitable for image classification because spatial features can be automatically extracted through convolutional layers. In

contrast, images are converted into one-dimensional vectors by ANN, causing important spatial relationships between pixels to be lost.

A test accuracy of 70.20% and a test loss of 0.9337 were achieved by the original CNN model. Although this result can be considered acceptable for a basic CNN architecture, further improvement was still required because the model did not generalize perfectly to unseen images. Therefore, an enhanced **new_CNN** model was developed using data augmentation, dropout, batch normalization, additional convolutional layers, and early stopping.

A test accuracy of 71.75% and a test loss of 0.8351 were achieved by the **new_CNN** model. Compared with the original CNN model, the test accuracy was increased by 1.55 percentage points, while the test loss was reduced by 0.0986. This indicates that slightly better performance was achieved by the enhanced model. Although the improvement was moderate, the lower test loss shows that prediction errors were reduced by the **new_CNN** model.

The improvement can be associated with the enhancement techniques applied in the **new_CNN** model. More image variations were learned through data augmentation, the training process was stabilized through batch normalization, overfitting was reduced through dropout, and deeper image features were extracted through the additional convolutional layer. However, weaker performance was still observed for some classes, such as cat, bird, deer, and dog. This may be due to the similar visual features of these animal classes in the low-resolution CIFAR-10 images.

In conclusion, the use of deep learning for image classification was successfully demonstrated by the CNN model, while a slight improvement in accuracy and loss was provided by the **new_CNN** model. The results show that classification performance can be improved through model enhancement techniques. However, further improvements can still be made through hyperparameter tuning, deeper architectures, or transfer learning.

9.0 Reflection

Name	Reflection
JOANNE CHING YIN XUAN	This tutorial provided valuable exposure to image classification and Convolutional Neural Networks (CNNs). A deeper understanding was developed regarding data preprocessing, feature normalization, model training, and performance evaluation using Python, TensorFlow, and Keras. The comparison between ANN and CNN models also highlighted the importance of spatial feature extraction in image classification tasks. A key challenge involved analyzing the performance of the developed models and interpreting the evaluation results obtained from the CIFAR-10 dataset. Particular attention was required to understand the performance gap between training and testing accuracy and to evaluate the limitations of the current CNN architecture. This challenge was addressed through detailed examination of the model outputs and comparison of ANN and CNN performance. Although the CNN model achieved satisfactory performance, several improvements could be made in future work. Techniques such as data augmentation, dropout layers, and batch normalization could be implemented to reduce overfitting and improve model performance. In addition, more evaluation metrics could be used to provide a better understanding of the model's classification results.
LEE YIN SHEN	Through this tutorial, the opportunity was given to have a hands-on experience with the application of deep learning in image classification problems. The entire workflow for preparing the image data, normalizing the pixel values, building the neural network models, training the models, and assessing their performance was investigated in the context of the CIFAR-10 classification task. It was also illustrated in the tutorial that the structure of a model can have a great influence on classification accuracy, particularly for image data. One of the important takeaways from this tutorial was the difference between an ANN and a CNN for image classification. The image was able to be classified by the ANN model, but the result was not as effective because the image data was flattened into a vector, causing spatial information to be lost. Better results were achieved by the CNN model due to the presence

	of the convolutional and pooling layers which better detected the visual patterns. It was demonstrated that CNN is more suitable for image-related tasks, as features are allowed to be learned directly from the structure of the image. The need for careful model performance assessment was also emphasized in the tutorial. The ANN model was outperformed by the CNN model, however, a slight difference between the training accuracy and testing accuracy was observed, which could be a sign of overfitting. It was demonstrated that a good training performance model is not necessarily a good model for unseen data. To this end, data augmentation, dropout, batch normalization, and early stopping are some possible techniques that can be explored for future improvement. The knowledge of using CNN for image classification was reinforced through this tutorial, and good practice on using deep learning concepts with Python, TensorFlow and Keras was given.
EVELYN GOH YUAN QI	Through the completion of this tutorial, a better understanding of image classification and Convolutional Neural Networks (CNNs) was developed. Practical experience was gained in image preprocessing, model training, prediction generation, and performance evaluation using Python and TensorFlow. In addition, the capability of CNNs to automatically extract image features and perform classification tasks was successfully demonstrated. Several challenges were encountered during the implementation process, particularly in configuring the required libraries and interpreting model outputs. These issues were resolved through troubleshooting and consultation of relevant documentation, resulting in a stronger understanding of the CNN workflow and evaluation process. Overall, valuable knowledge and practical skills related to deep learning and image classification were acquired. The results also highlighted the importance of model evaluation and the need to address potential overfitting issues. Future improvements could be achieved through techniques such as data augmentation, dropout layers, and hyperparameter tuning to further enhance model performance.